

IF3270 - Pembelajaran Mesin

Tugas Besar 1

Feedforward Neural Network



Disusun oleh:

Sebastian Hung Yansen 13523070

Aramazaya 13523082

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESA 10, BANDUNG 40132**

2026

DAFTAR ISI

DAFTAR ISI.....	1
BAB I DESKRIPSI PERSOALAN.....	2
1.1. Tujuan.....	2
1.2. Spesifikasi.....	2
BAB II PEMBAHASAN.....	3
2.1. Penjelasan Implementasi.....	3
2.1.1. Deskripsi Kelas.....	4
2.1.2. Forward Propagation.....	13
2.1.3. Backward Propagation.....	13
2.1.4. Adam Optimizer.....	14
2.2. Hasil Pengujian.....	16
2.2.1. Pengaruh Depth dan Width.....	16
2.2.2. Pengaruh Fungsi Aktivasi.....	18
2.2.3. Pengaruh Learning Rate.....	20
2.2.4. Pengaruh Inisialisasi Bobot.....	23
2.2.5. Pengaruh Regularisasi.....	24
2.2.6. Perbandingan Dengan Library sklearn.....	26
2.2.7. Perbandingan Gradient Descent Standar Dengan Adam.....	26
BAB III KESIMPULAN DAN SARAN.....	28
3.1. Kesimpulan.....	28
3.2. Saran.....	28
REFERENSI.....	29
LAMPIRAN.....	30

BAB I

DESKRIPSI PERSOALAN

1.1. Tujuan

Tugas Besar I pada kuliah IF3270 Pembelajaran Mesin agar peserta kuliah mendapatkan wawasan tentang bagaimana cara mengimplementasikan Feedforward Neural Network (FFNN). Pada tugas ini, peserta kuliah akan ditugaskan untuk mengimplementasikan FFNN from scratch.

1.2. Spesifikasi

Diimplementasikan suatu modul FFNN yang memenuhi ketentuan-ketentuan berikut:

1. Dapat menerima jumlah neuron dari tiap layer termasuk input dan output layer.
2. Dapat menerima fungsi aktivasi dari tiap layer.
3. Dapat menerima fungsi loss dari model tersebut.
4. Memiliki mekanisme untuk inisialisasi bobot tiap neuron termasuk bias.
5. Instance model harus bisa menyimpan bobot setiap neuron termasuk bias.
6. Instance model harus bisa menyimpan gradien bobot setiap neuron termasuk bias.
7. Instance model memiliki method untuk menampilkan distribusi bobot dari setiap layer.
8. Instance model memiliki method untuk menampilkan distribusi gradien bobot dari setiap layer.
9. Instance model memiliki method untuk save dan load.
10. Model memiliki implementasi forward propagation dengan ketentuan dapat menerima input berupa batch.
11. Model memiliki implementasi backward propagation untuk menghitung perubahan gradien.
12. Model memiliki implementasi metode regularisasi L1 dan L2.
13. Model memiliki implementasi weight update dengan menggunakan gradient descent untuk memperbarui bobot berdasarkan gradien yang telah dihitung.
14. Implementasi pelatihan model dapat menerima parameter batch size, learning rate, jumlah epoch, dan verbose.

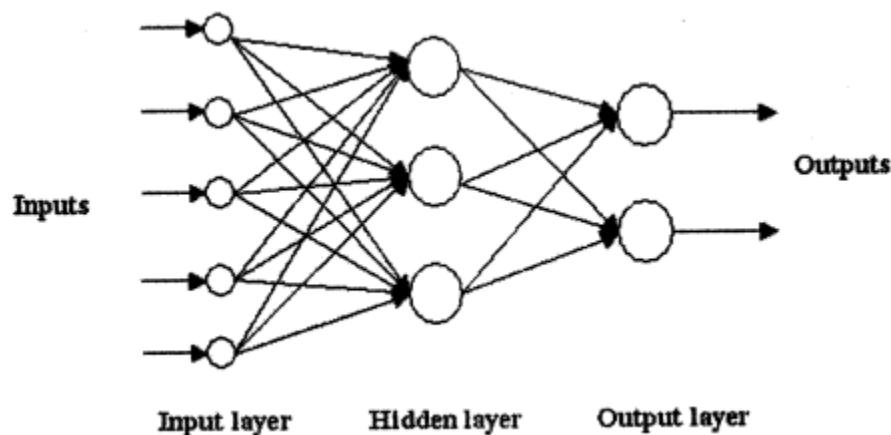
BAB II PEMBAHASAN

2.1. Penjelasan Implementasi

Artificial neural networks pada dasarnya terinspirasi dari otak dan sistem saraf. Kemampuan otak manusia jauh melebihi kemampuan komputer biasa. Ciri khas paling penting dari sebuah otak adalah kemampuannya untuk belajar dan beradaptasi, sebuah kemampuan yang komputer tidak miliki. Sebuah komputer hanya menyelesaikan tugas-tugas tertentu berdasarkan instruksi dan program yang ada dalam komputernya.

Neural network dibangun atas banyak neuron yang saling berhubungan melalui *weights*. Sebuah neuron dapat dipahami sebagai sebuah *processing unit*. Setiap neuron menerima informasi melalui hubungan *weights* dan menghasilkan sebuah output melalui fungsi aktivasi. Oleh karena itu, ciri khas *neural network* yang membedakannya dari komputer biasa adalah kemampuannya untuk *learning*. Sebuah *neural network* mampu mempelajari lingkungannya dan meningkatkan kinerjanya melalui *learning*. Terdapat dua kategori arsitektur network: *recurrent neural networks* dan *feedforward neural networks*.

Feedforward Neural Network (FFNN) adalah kategori *neural network* di mana tidak ada feedback dari output neuron terhadap input neuron lain. FFNN langsung memberikan hasil output neuron ke input neuron lainnya. Singkatnya, informasi mengalir melalui network dari input ke output untuk menghitung prediksi.



Sebuah network dipisah-pisah menggunakan lapisan atau *layer*. Pada gambar, terlihat ada *hidden layer* di antara input dan output layer. Fungsi *hidden layer* yakni untuk intervensi informasi di antara *input layer* dan *output layer*. Hal tersebut memungkinkan *network* untuk mengekstraksi statistika tingkat tinggi.

Hasil implementasi *Feedforward Neural Network from scratch* sebagai berikut:

2.1.1. Deskripsi Kelas

Didefinisikan beberapa kelas yakni kelas Value, Layer, dan FeedForwardNN. Kelas Value merupakan kelas nilai skalar yang men-*track* operasi untuk diferensiasi otomatis. Untuk implementasi kelas Value sebagai berikut:

```
class Value:
    def __init__(self, data, _prev=None, _op=''):
        self.data = data
        self.gradient = 0.0
        self._back = lambda: None
        self._prev = _prev if _prev is not None else set()
        self._op = _op
        self._layer : List[Value] = []

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, _prev={self, other}, _op='+')
        def _back():
            self.gradient += out.gradient
            other.gradient += out.gradient
        out._back = _back
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, _prev={self, other}, _op='*')
        def _back():
            self.gradient += other.data * out.gradient
            other.gradient += self.data * out.gradient
        out._back = _back
        return out

    def __pow__(self, other):
        out = Value(self.data ** other, _prev={self}, _op='**')
        def _back():
            self.gradient += other * (self.data ** (other - 1)) * out.gradient
        out._back = _back
        return out

    def __neg__(self):
        return self * -1

    def __radd__(self, other):
        return self + other

    def __sub__(self, other):
        return self + (-other)
```

```

def __rsub__(self, other):
    return other + (-self)

def __rmul__(self, other):
    return self * other

def sigmoid(self):
    s = 1 / (1 + np.exp(-self.data))
    out = Value(s, _prev={self}, _op='sigmoid')
    def _back():
        self.gradient += s * (1 - s) * out.gradient
    out._back = _back
    return out

def relu(self):
    out = Value(self.data if self.data > 0 else 0, _prev={self}, _op='relu')
    def _back():
        self.gradient += (out.data > 0) * out.gradient
    out._back = _back
    return out

def tanh(self):
    t = np.tanh(self.data)
    out = Value(t, _prev={self}, _op='tanh')
    def _back():
        self.gradient += (1 - t ** 2) * out.gradient
    out._back = _back
    return out

def linear(self):
    out = Value(self.data, _prev={self}, _op='linear')
    def _back():
        self.gradient += out.gradient
    out._back = _back
    return out

def log(self):
    out = Value(np.log(self.data + 1e-7), _prev={self}, _op='log')
    def _back():
        self.gradient += (1 / (self.data + 1e-7)) * out.gradient
    out._back = _back
    return out

def abs(self):
    out = Value(abs(self.data), _prev={self}, _op='abs')
    def _back():
        self.gradient += (1.0 if self.data > 0 else -1.0) * out.gradient
    out._back = _back
    return out

def exp(self):
    out = Value(np.exp(self.data), _prev={self}, _op='exp')
    def _back():
        self.gradient += out.data * out.gradient
    out._back = _back
    return out

def leaky_relu(self, alpha=0.01):
    out = Value(self.data if self.data > 0 else alpha * self.data, _prev={self},

```

```

_op='leaky_relu')
    def _back():
        self.gradient += (1.0 if self.data > 0 else alpha) * out.gradient
    out._back = _back
    return out

    def elu(self, alpha=1.0):
        out = Value(self.data if self.data > 0 else alpha * (np.exp(self.data) - 1),
_prev={self}, _op='elu')
        def _back():
            self.gradient += (1.0 if self.data > 0 else out.data + alpha) * out.gradient
        out._back = _back
        return out

    def softmax(self):
        layer = self._layer
        exps = [np.exp(neuron.data) for neuron in layer]
        sum_exps = sum(exps)
        proba = [e / sum_exps for e in exps]
        i = layer.index(self)
        out = Value(proba[i], _prev=set(layer), _op='softmax')
        def _back():
            for j, neuron in enumerate(layer):
                if j == i:
                    neuron.gradient += proba[i]*(1-proba[i]) * out.gradient
                else:
                    neuron.gradient += -proba[i]*proba[j] * out.gradient
        out._back = _back
        return out

    def back(self):
        topo = []
        visited = set()
        def build_topo(v):
            if v not in visited:
                visited.add(v)
                for child in v._prev:
                    build_topo(child)
            topo.append(v)
        build_topo(self)
        self.gradient = 1.0
        for node in reversed(topo):
            node._back()

    def clear_graph(self):
        visited = set()
        stack = [self]
        while stack:
            node = stack.pop()
            if id(node) in visited:
                continue
            visited.add(id(node))
            stack.extend(node._prev)
            node._back = lambda: None
            node._prev = set()

```

Kelas Value menyimpan sebuah *value data* beserta gradiennya, reference node *Value* sebelumnya, dan operasi yang membuat *value* tersebut. Kelas value juga mengimplementasikan

operasi-operasi aritmatika dan fungsi-fungsi menyerupai aktivasi seperti sigmoid, relu, tanh, linear, log, abs, exp, leaky_relu, elu, dan softmax. Masing-masing fungsi tersebut memiliki metode `_back` untuk mendefinisikan bagaimana gradien merambat. Metode `clear_graph` digunakan untuk membersihkan graf.

Kelas `Layer` adalah merepresentasikan lapisan yang dibangun di atas objek `Value`. Implementasi kelas `Layer` sebagai berikut:

```
class Layer:
    def __init__(self, input_size: int, output_size: int, activation_function: str = "",
    _V=None):
        self.input_size = input_size
        self.output_size = output_size
        self.activation_function = activation_function
        self.weights = []
        self.biases = []
        self._V = _V or Value

    def init_weights(self, method: str, lower_bound: float = 0.0, upper_bound: float = 1.0,
    mean: float = 0.0, variance: float = 1.0, seed: int = 42) -> None:
        if method == "zeros":
            w = np.zeros((self.input_size, self.output_size))
            b = np.zeros(self.output_size)
        elif method == "random":
            w = np.random.uniform(lower_bound, upper_bound, (self.input_size,
            self.output_size))
            b = np.random.uniform(lower_bound, upper_bound, self.output_size)
        elif method == "normal":
            w = np.random.normal(mean, variance, (self.input_size, self.output_size))
            b = np.random.normal(mean, variance, self.output_size)
        elif method == "xavier":
            limit = np.sqrt(6 / (self.input_size + self.output_size))
            w = np.random.uniform(-limit, limit, (self.input_size, self.output_size))
            b = np.random.uniform(-limit, limit, self.output_size)
        elif method == "he":
            stddev = np.sqrt(2 / self.input_size)
            w = np.random.normal(0, stddev, (self.input_size, self.output_size))
            b = np.random.normal(0, stddev, self.output_size)
        else:
            print(f"Unknown weight initialization method '{method}'. Defaulting to zeros.")
            w = np.zeros((self.input_size, self.output_size))
            b = np.zeros(self.output_size)
        self.weights = [[self._V(w[i][j]) for j in range(self.output_size)] for i in
        range(self.input_size)]
        self.biases = [self._V(b[j]) for j in range(self.output_size)]

    def forward(self, x: List) -> List:
        z = [sum(x[i]*self.weights[i][j] for i in range(self.input_size)) + self.biases[j]
        for j in range(self.output_size)]
        if self.activation_function == "softmax":
            for node in z:
                node._layer = z
            act = getattr(self._V, self.activation_function)
            return [act(val) for val in z]
```


Kelas Layer menyimpan konfigurasi layer seperti input_size, output_size, serta nama fungsi aktivasi yang di-*apply*. Prosedur init_weights menginisialisasi weights dan bias menggunakan metode yang dipilih antara Zeros, Random, Normal, Xavier, dan He. Fungsi forward menghitung z untuk setiap neuron dan meng-*apply* fungsi aktivasinya.

Kelas FeedForwardNN merupakan kelas utama Feed Forward Neural network. Kelas ini mengelola training, perhitungan loss, dan optimisasi. Implementasinya sebagai berikut:

```
class FeedForwardNN:
    """Main class for the FFNN.

    Parameters
    -----
    neurons : num of neurons per each layer -- Int List
    activation_function (linear) : activation function for each layer starting with first
hidden layer-- String List
    loss_function (mse) : loss function to optimize -- String
    learning_rate (0.1) : learning rate for gradient descent -- Float
    loss_threshold (0.01) : threshold for early stopping -- Float
    epochs (100) : maximum number of epochs -- Int
    seed (42) : random seed for weight initialization -- Int
    weight_init (zeros) : method for weight initialization (zeros, random, normal) default
zeros -- String
    weight_ub (1.0) : upper bound for random weight initialization -- Float
    weight_lb (0.0) : lower bound for random weight initialization -- Float
    weight_mean (0.0) : mean for normal weight initialization -- Float
    weight_variance (1.0) : variance for normal weight initialization -- Float
    """
    def __init__(self, neurons: List[int], activation_function: List[str] = [],
        loss_function: str = 'mse', learning_rate: float = 0.1, loss_threshold:
float = 0.01,
        epochs: int = 100, seed: int = 42, weight_init: str = "zeros",
        weight_ub: float = 1.0, weight_lb: float = 0.0, weight_mean: float = 0.0,
        weight_variance: float = 1.0, batch_size: int = 32, verbose: bool = True,
        l1: float = 0.0, l2: float = 0.0, optimizer: str = 'sgd',
        adam_beta1: float = 0.9, adam_beta2: float = 0.999, adam_epsilon: float =
1e-8):
        self._V = Value
        self.neurons = neurons
        if not activation_function:
            self.activation_function = ['linear' for _ in range(len(neurons) - 1)]
        elif len(activation_function) != len(neurons) - 1:
            self.activation_function = activation_function + ['linear'] * (len(neurons) - 1
- len(activation_function))
        else:
            self.activation_function = activation_function
        self.loss_function = loss_function
        self.learning_rate = learning_rate
        self.loss_threshold = loss_threshold
```

```

self.epochs = epochs
self.X = np.array([])
self.y = np.array([])
self.verbose = verbose
self.batch_size = batch_size
self.l1 = l1
self.l2 = l2
self._adam = AdamOptimizer(alpha=learning_rate, beta1=adam_beta1, beta2=adam_beta2,
epsilon=adam_epsilon) if optimizer == 'adam' else None
self.input_layer = Layer(neurons[0], neurons[1], self.activation_function[0],
_V=self._V)
self.hidden_layers = [Layer(neurons[i], neurons[i+1], self.activation_function[i],
_V=self._V) for i in range(1, len(neurons)-2)]
self.output_layer = Layer(neurons[-2], neurons[-1], self.activation_function[-1],
_V=self._V)
self.res : List[List] = []
self.loss = self._V(0.0)
self.history: Dict[str, List[float]] = {"train_loss": [], "val_loss": []}
self._initialize_weights(weight_init, weight_lb, weight_ub, weight_mean,
weight_variance, seed)

def _initialize_weights(self, method : str, lower_bound: float = 0.0, upper_bound:
float = 1.0, mean: float = 0.0, variance : float = 1.0, seed : int = 42) -> None:
    np.random.seed(seed)
    for layer in [self.input_layer] + self.hidden_layers + [self.output_layer]:
        layer.init_weights(method, lower_bound, upper_bound, mean, variance, seed)

def fit(self, X: np.ndarray, y: np.ndarray, X_val: Optional[np.ndarray] = None, y_val:
Optional[np.ndarray] = None) -> Dict[str, List[float]]:
    self.X = X
    self.y = y
    self.history = {"train_loss": [], "val_loss": []}
    self._train(X_val=X_val, y_val=y_val)
    return self.history

def _compute_validation_loss(self, X_val: np.ndarray, y_val: np.ndarray) -> float:
    old_res, old_y = self.res, self.y
    self.res = []
    self.y = np.asarray(y_val)
    for idx in range(X_val.shape[0]):
        x_input = [self._V(X_val[idx, j]) for j in range(X_val.shape[1])]
        self._forward_propagation(x_input)
    loss_val = self._compute_loss(0).data
    self.loss.clear_graph()
    self.res = old_res
    self.y = old_y
    return loss_val

def _train(self, X_val: Optional[np.ndarray] = None, y_val: Optional[np.ndarray] =
None) -> None:
    for epoch in range(self.epochs):
        batch = 0
        epoch_loss = 0.0
        n_batches = 0
        while batch*self.batch_size < self.X.shape[0]:
            for i in range(self.batch_size):
                idx = i + batch * self.batch_size
                if idx >= self.X.shape[0]:
                    break
                x_input = [self._V(self.X[idx, j]) for j in range(self.X.shape[1])]

```

```

        self._forward_propagation(x_input)
        self.loss = self._compute_loss(batch * self.batch_size)
        epoch_loss += self.loss.data
        n_batches += 1
        self._backward_propagation()
        batch += 1
        self.res = []
    avg_loss = epoch_loss / n_batches
    self.history["train_loss"].append(avg_loss)
    msg = f'Epoch {epoch+1}/{self.epochs}, Loss: {avg_loss:.6f}'
    if X_val is not None and y_val is not None:
        val_loss = self._compute_validation_loss(X_val, y_val)
        self.history["val_loss"].append(val_loss)
        msg += f', Val Loss: {val_loss:.6f}'
    if self.verbose:
        print(msg)

    if avg_loss < self.loss_threshold:
        print('Loss threshold reached. Stopping training.')
        break

def _compute_loss(self, offset: int = 0):
    V = self._V
    total = V(0.0)
    if self.loss_function == 'mse':
        for i in range(len(self.res)):
            y_i = self.y[i+offset]
            for j in range(len(self.res[i])):
                target = float(y_i[j]) if hasattr(y_i, '__len__') else float(y_i)
                total += (self.res[i][j] - V(target)) ** 2
            total = total * (1 / len(self.res))
    elif self.loss_function == "binary_cross_entropy":
        for i in range(len(self.res)):
            y_i = self.y[i+offset]
            if not hasattr(y_i, '__len__'):
                target = [1.0 - float(y_i), float(y_i)]
            else:
                target = y_i
            for j in range(len(self.res[i])):
                p = self.res[i][j]
                y = float(target[j])
                total += -(V(y) * p.log() + V(1 - y) * (V(1.0) - p).log())
            total = total * (1 / len(self.res))
    elif self.loss_function == "categorical_cross_entropy":
        n_classes = len(self.res[0])
        for i in range(len(self.res)):
            y_i = self.y[i+offset]
            if not hasattr(y_i, '__len__'):
                target = [1.0 if j == int(y_i) else 0.0 for j in range(n_classes)]
            else:
                target = y_i
            for j in range(len(self.res[i])):
                total += -(V(float(target[j])) * self.res[i][j].log())
            total = total * (1 / len(self.res))
    else:
        print(f"Unknown loss function '{self.loss_function}'. Defaulting to MSE.")
        for i in range(len(self.res)):
            for j in range(len(self.res[i])):
                total += (self.res[i][j] - V(float(self.y[i+offset][j]))) ** 2
            total = total * (1 / len(self.res))

```

```

    all_layers = [self.input_layer] + self.hidden_layers + [self.output_layer]
    for layer in all_layers:
        for row in layer.weights:
            for w in row:
                if self.l1 > 0:
                    total += V(self.l1) * w.abs()
                if self.l2 > 0:
                    total += V(self.l2) * w ** 2
    return total

def compute_gradients(self, X: np.ndarray, y: np.ndarray) -> List[Dict[str,
np.ndarray]]:
    old_res, old_y = self.res, self.y
    self.res = []
    self.y = np.asarray(y)
    for idx in range(X.shape[0]):
        x_input = [self._V(X[idx, j]) for j in range(X.shape[1])]
        self._forward_propagation(x_input)
    self.loss = self._compute_loss(0)
    self.loss.back()
    result = self.get_weights()
    self.loss.clear_graph()
    all_layers = [self.input_layer] + self.hidden_layers + [self.output_layer]
    for layer in all_layers:
        for row in layer.weights:
            for w in row:
                w.gradient = 0.0
        for b in layer.biases:
            b.gradient = 0.0
    self.res = old_res
    self.y = old_y
    return result

def get_weights(self) -> List[Dict[str, np.ndarray]]:
    result = []
    all_layers = [self.input_layer] + self.hidden_layers + [self.output_layer]
    for l_idx, layer in enumerate(all_layers):
        result.append({
            "layer": l_idx,
            "weights": np.array([[w.data for w in row] for row in layer.weights]),
            "biases": np.array([b.data for b in layer.biases]),
            "weight_gradients": np.array([[w.gradient for w in row] for row in
layer.weights]),
            "bias_gradients": np.array([b.gradient for b in layer.biases]),
        })
    return result

def inspect(self, x_row: np.ndarray) -> None:
    self.res = []
    x_input = [self._V(x_row[j]) for j in range(len(x_row))]
    self._forward_propagation(x_input)
    print("Raw output values:", [round(v.data, 6) for v in self.res[-1]])

def predict(self, X: np.ndarray) -> np.ndarray:
    predictions = []
    self.res = []
    for i in range(X.shape[0]):
        x_input = [self._V(X[i, j]) for j in range(X.shape[1])]
        self._forward_propagation(x_input)
        predictions.append([neuron.data for neuron in self.res[-1]])

```

```

arr = np.array(predictions)
if arr.shape[1] == 1:
    return arr.flatten()
return np.argmax(arr, axis=1).astype(int)

def save(self, path: str) -> None:
    import json
    all_layers = [self.input_layer] + self.hidden_layers + [self.output_layer]
    weights_data = []
    for layer in all_layers:
        weights_data.append({
            "weights": [[w.data for w in row] for row in layer.weights],
            "biases": [b.data for b in layer.biases]
        })
    model_data = {
        "neurons": self.neurons,
        "activation_function": self.activation_function,
        "loss_function": self.loss_function,
        "learning_rate": self.learning_rate,
        "loss_threshold": self.loss_threshold,
        "epochs": self.epochs,
        "batch_size": self.batch_size,
        "l1": self.l1,
        "l2": self.l2,
        "optimizer": "adam" if self._adam is not None else "sgd",
        "adam_beta1": self._adam.beta1 if self._adam else 0.9,
        "adam_beta2": self._adam.beta2 if self._adam else 0.999,
        "adam_epsilon": self._adam.epsilon if self._adam else 1e-8,
        "weights": weights_data,
        "history": self.history,
    }
    with open(path, "w") as f:
        json.dump(model_data, f, indent=2)

@classmethod
def load(cls, path: str) -> "FeedForwardNN":
    import json
    with open(path, "r") as f:
        d = json.load(f)
    model = cls(
        neurons=d["neurons"],
        activation_function=d["activation_function"],
        loss_function=d["loss_function"],
        learning_rate=d["learning_rate"],
        loss_threshold=d["loss_threshold"],
        epochs=d["epochs"],
        batch_size=d["batch_size"],
        l1=d["l1"],
        l2=d["l2"],
        optimizer=d["optimizer"],
        adam_beta1=d["adam_beta1"],
        adam_beta2=d["adam_beta2"],
        adam_epsilon=d["adam_epsilon"],
    )
    all_layers = [model.input_layer] + model.hidden_layers + [model.output_layer]
    for layer, layer_data in zip(all_layers, d["weights"]):
        for i, row in enumerate(layer.weights):
            for j, w in enumerate(row):
                w.data = layer_data["weights"][i][j]
        for j, b in enumerate(layer.biases):

```

```
b.data = layer_data["biases"][j]
model.history = d.get("history", {"train_loss": [], "val_loss": []})
return model
```

`__init__` menyimpan list neuron dan hyperparameter, membuat layer-layer, dan menginisialisasi *weights*. Terdapat juga setup untuk Adam Optimizer tetapi hal tersebut bersifat opsional. Fungsi `fit` menyimpan data training dan memanggil fungsi `_train`, fungsi yang melakukan training mini-batch pada epochs. Fungsi `_train` juga membangun *graph* komputasional untuk setiap sampel dalam batch melalui fungsi `_forward_propagation`. Loss juga dihitung dengan fungsi `_compute_loss` untuk batch serta dipantau apabila loss di bawah threshold. Update parameter dilakukan dengan `_backward_propagation` untuk update parameter. Apabila semua proses sudah selesai, *graph* di bersihkan dengan fungsi `clear_graph` dari Value untuk menghindari *memory leak* dan memastikan graf lama tidak digunakan lagi. Fungsi `predict` digunakan untuk melakukan forward propagation setiap row di *X* dan mengembalikan outputnya. Method `save` dan `load` digunakan untuk menyimpan dan memuat model FFNN yang sudah di-*train*. Tidak hanya itu, method juga menyimpan serta memuat semua layer dan data bobotnya.

2.1.2. Forward Propagation

Forward Propagation adalah proses membawa data pada input melewati setiap neuron pada hidden layer hingga mencapai output layer. Implementasinya sendiri sebagai berikut:

```
def _forward_propagation(self, x_input) -> None:
    out = self.input_layer.forward(x_input)
    for layer in self.hidden_layers:
        out = layer.forward(out)
    out = self.output_layer.forward(out)
    self.res.append(out)
```

Fungsi bekerja dengan cara mem-*push* satu input vector ke dalam network dan menyimpan output dalam `self.res`. Perlu diperhatikan terdapat fungsi `forward` yang sempat disinggung pada penjelasan kelas `FeedForwardNN`.

2.1.3. Backward Propagation

Backward Propagation adalah algoritma pelatihan FFNN dengan cara menghitung *gradient error* terhadap setiap bobot menggunakan chain rule. Dari forward propagation, angka error dapat dihitung. Angka error tersebut kemudian akan digunakan backward propagation untuk meng-*update* setiap weight dan bias dengan learning tertentu. Proses tersebut akan diulang

terus hingga mendapatkan nilai weight dan bias yang dapat memberikan nilai error sekecil mungkin pada output layer. Implementasi backward propagation sendiri sebagai berikut:

```
def _backward_propagation(self) -> None:
    self.loss.back()
    all_layers = [self.input_layer] + self.hidden_layers + [self.output_layer]
    if self._adam is not None:
        all_params = []
        all_grads = []
        for layer in all_layers:
            for neuron in layer.weights:
                for w in neuron:
                    all_params.append(w.data)
                    all_grads.append(w.gradient)
            for bias in layer.biases:
                all_params.append(bias.data)
                all_grads.append(bias.gradient)
        updated = self._adam.step(np.array(all_params), np.array(all_grads))
        idx = 0
        for layer in all_layers:
            for neuron in layer.weights:
                for w in neuron:
                    w.data = updated[idx]
                    w.gradient = 0.0
                    idx += 1
            for bias in layer.biases:
                bias.data = updated[idx]
                bias.gradient = 0.0
                idx += 1
    else:
        for layer in all_layers:
            for neuron in layer.weights:
                for weight in neuron:
                    weight.data -= self.learning_rate * weight.gradient
                    weight.gradient = 0.0
            for bias in layer.biases:
                bias.data -= self.learning_rate * bias.gradient
                bias.gradient = 0.0
    self._clear_graph()
```

Fungsi backward propagation sendiri *trigger* ketika loss. Selanjutnya fungsi akan meng-*apply* update SGD menggunakan gradient setiap parameter. Jika menggunakan Adam, semua weight dan bias diubah menjadi array, eksekusi step pada Adam, dan menulis value yang sudah di-update. Perlu diperhatikan terdapat beberapa fungsi yang sudah sempat disinggung pada penjelasan kelas FeedForwardNN

2.1.4. Adam Optimizer

Adam merupakan algoritma untuk *gradient-based optimization* orde pertama dari *stochastic objective functions*, berdasarkan estimasi adaptif dari momen orde rendah. Berdasarkan paper yang menjelaskan tentang algoritma adam, algoritmanya efisien secara komputasi, memiliki

sedikit kebutuhan memori, tidak berubah terhadap penskalaan diagonal gradien, dan sangat cocok untuk masalah yang besar dalam hal data dan/atau parameter. Implementasinya juga sederhana dan mengikuti pseudocode yang diberikan paper sebagai berikut:

```
import numpy as np

class AdamOptimizer:
    def __init__(self, alpha=0.001, beta1=0.9, beta2=0.999, epsilon=1e-8):
        """
        Args:
            alpha (float): step size (learning rate)
            beta1 (float): Exponential decay rate for first moment, in [0, 1)
            beta2 (float): Exponential decay rate for second moment, in [0, 1)
            epsilon (float): Small constant for numerical stability
        """
        self.alpha = alpha
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon
        self.t = 0 # initialize timestep
        self.m = None # initialize first moment vector
        self.v = None # initialize second raw moment vector

    def step(self, theta, gt):
        # Initialize m and v
        if self.m is None:
            self.m = np.zeros_like(theta)
            self.v = np.zeros_like(theta)

        self.t += 1

        # get gradients

        # Update biased first moment estimate
        self.m = self.beta1 * self.m + (1.0 - self.beta1) * gt

        # Update biased second raw moment estimate
        self.v = self.beta2 * self.v + (1.0 - self.beta2) * (gt * gt)

        # Compute bias-corrected first moment estimate
        m_hat = self.m / (1.0 - self.beta1 ** self.t)

        # Compute bias-corrected second raw moment estimate
        v_hat = self.v / (1.0 - self.beta2 ** self.t)

        # Update parameters
        theta_updated = theta - self.alpha * m_hat / (np.sqrt(v_hat) + self.epsilon)

        return theta_updated
```

Step diimplementasikan sesuai dengan algoritma pseudocode dari paper. Untuk pengulangannya dipanggil langsung dari notebook saat pengujian

2.2. Hasil Pengujian

Dataset yang digunakan untuk pengujian adalah dataset placement ranking universitas berdasarkan ranking, internship, jurusan, dan lain-lain. Dataset sudah disediakan dari asisten.

Mayoritas pengujian ditetapkan 1 input layer, 2 hidden layer, 1 output layer. Fungsi aktivasi yang ditetapkan linear, tanh, leaky_relu, sigmoid. Loss function ditetapkan binary_cross_entropy. Didapatkan hasil pengujian sebagai berikut:

2.2.1. Pengaruh Depth dan Width

Didapatkan hasil akurasi dari variasi depth dan width sebagai berikut:

Depth

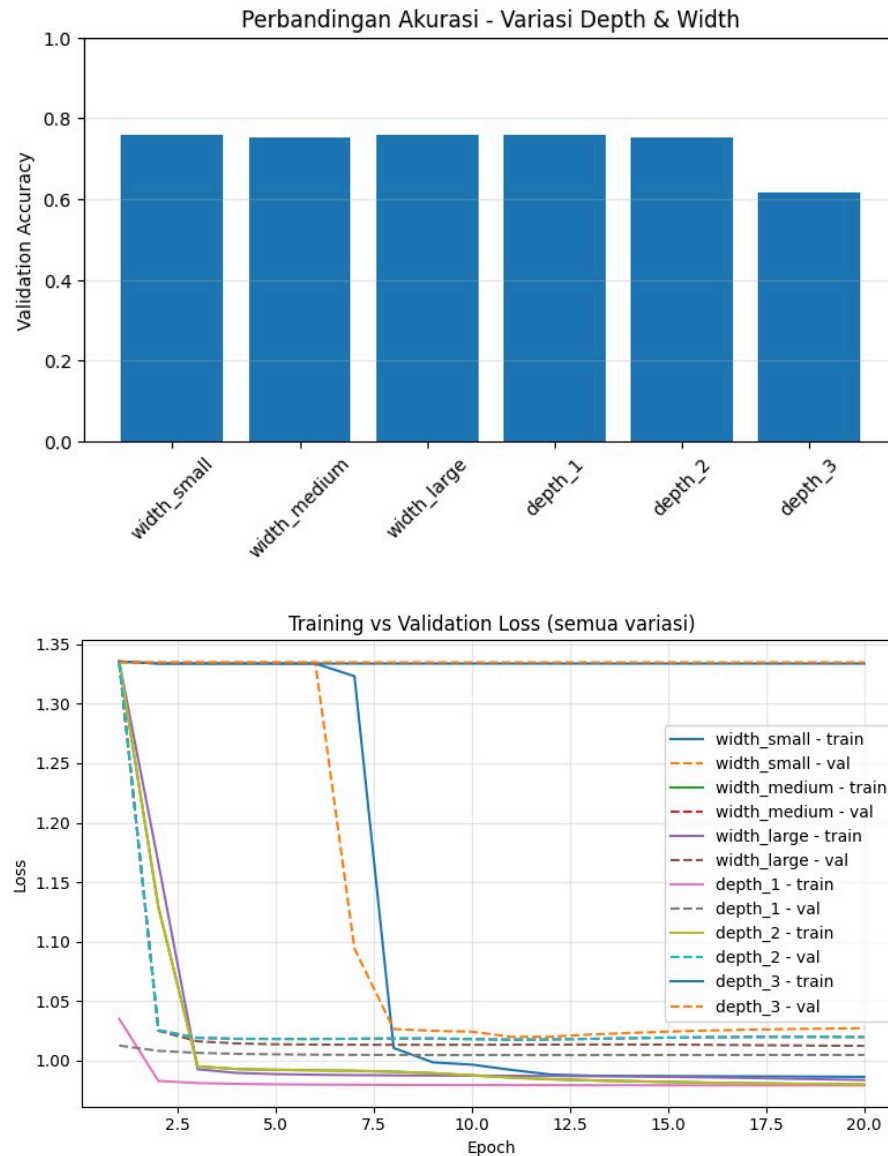
Neuron	Akurasi
HL1(4)	0.7590
HL1(4) + HL2(4)	0.7540
HL1(4) + HL2(4) + HL3(4)	0.6155

Width

Neuron	Akurasi
HL1(2) + HL2(2)	0.7580
HL1(4) + HL2(4)	0.7540
HL1(8) + HL2(8)	0.7585

Perbedaan angka akurasi antar variasi tidak begitu jauh. Terlihat bahwa 1 hidden layer dengan 4 neuron menghasilkan akurasi paling tinggi di angka 0.7590. Pada variasi 3 Layer 4 neuron masing-masing didapatkan angka akurasi 0.6155. Berdasarkan pengujian berulang kali, hasil angka tersebut merupakan hasil ketika model memprediksi semua data 'placed'. Hal tersebut terjadi karena mereka menetapkan semua status data berdasarkan mayoritas atau rata-rata status semua data. Kondisi tersebut merupakan kondisi Collapse dan bisa terjadi akibat model yang gagal di-*train*.

Didapatkan grafik perbandingan akurasi dan grafik training loss dan validation loss sebagai berikut:



Hampir semua proses training semua model berjalan dengan lancar karena mampu konvergen di epoch awal. Satu model yang telat konvergen adalah model dengan variasi width kecil. Satu model lagi yang gagal konvergen adalah model dengan depth 3 hidden layer. Artinya, model tersebut gagal total sehingga angka loss tetap sangat besar hingga akhir epoch 20.

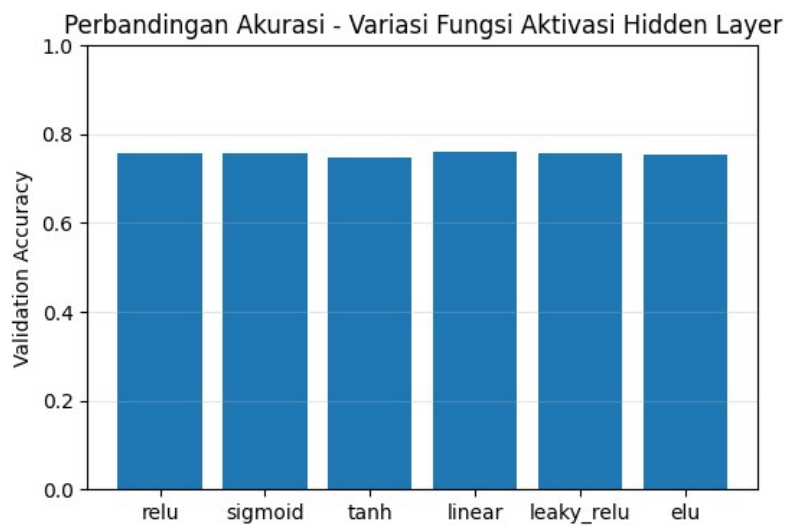
2.2.2. Pengaruh Fungsi Aktivasi

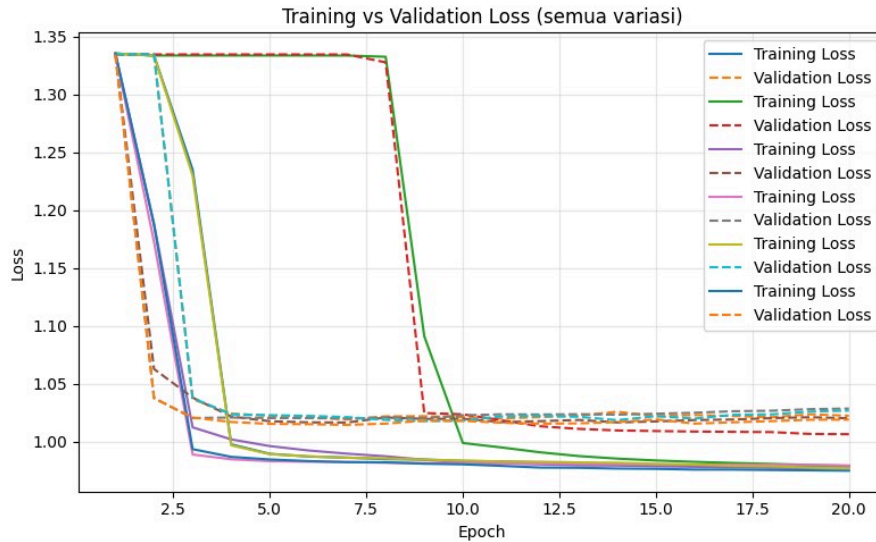
Pada pengujian fungsi aktivasi ditetapkan 3 layer dengan variasi fungsi aktivasi di layer pertama. Kedua layer lainnya ditetapkan fungsi aktivasi ReLU dan Sigmoid, didapatkan hasil akurasi sebagai berikut:

Fungsi Aktivasi	Akurasi
ReLU	0.7575
Sigmoid	0.7555
tanh	0.7470
linear	0.7590
leaky_relu	0.7565
elu	0.7540

Terlihat bahwa fungsi aktivasi linear memberikan akurasi paling tinggi sedangkan fungsi aktivasi tanh memberikan akurasi paling rendah. Hal tersebut menunjukkan bahwa dataset kemungkinan besar memiliki hubungan linear yang kuat. Akibatnya, penggunaan fungsi yang non-linier malah membuat data lebih kompleks padahal tidak perlu.

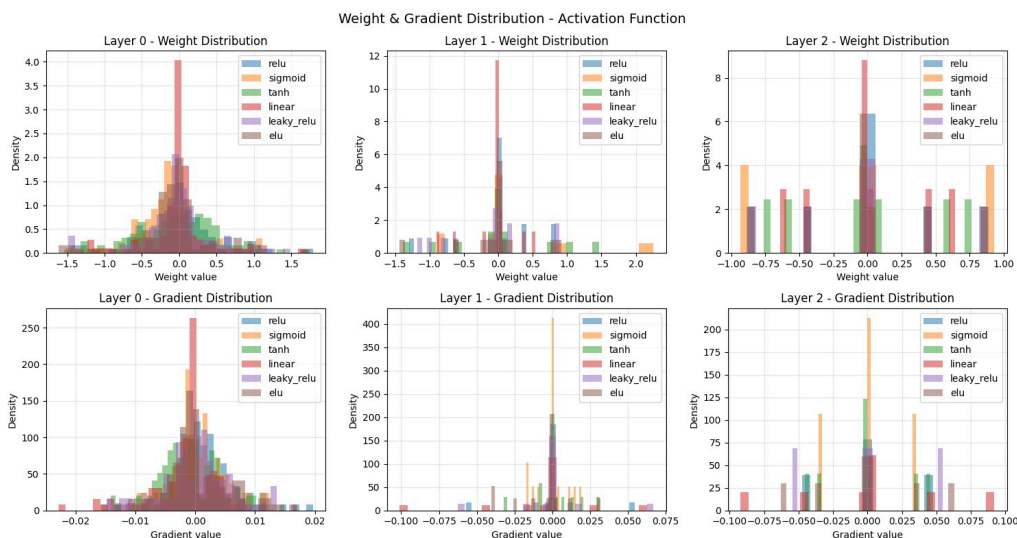
Didapatkan grafik perbandingan akurasi dan grafik training loss dan validation loss sebagai berikut:





Secara berurut legenda dari atas ke bawah: ReLU, Sigmoid, tanh, linear, leaky_relu, elu. Secara garis besar, proses training semua model berjalan dengan lancar karena mampu konvergen di epoch-epoch 2-3. Satu model yang telat konvergen di epoch 8 adalah model dengan fungsi aktivasi Sigmoid. Walau pada akhirnya loss pada semua model hampir sama semua, model dengan fungsi aktivasi Sigmoid membutuhkan waktu lebih lama untuk mencapai konvergen. Apabila proses training dilakukan dengan epoch yang lebih sedikit, ada kemungkinan model dengan fungsi aktivasi Sigmoid gagal konvergen.

Selain itu, didapatkan grafik distribusi bobot dan gradien bobot dari beberapa/semua layer pada model sebagai berikut:



Terlihat bahwa *weight distribution* di layer 2 lebih merata dan menyebar dibandingkan layer 0 dan 1 untuk model dengan fungsi aktivasi. Namun, distribusi bobot tidak merata seluruhnya. Sebagian besar model berdiam di angka 0 yang menandakan bahwa banyak bobot yang tidak berkontribusi. *Density* yang tinggi juga menandakan bahwa sangat banyak bobot yang tidak berkontribusi tersebut.

Gradient distribution cukup merata di layer 2 untuk semua fungsi aktivasi. Learning rate 0.001 berdiam di kisaran angka 0. Sama seperti pada grafik *weight distribution*, banyak model yang berdiam di kisaran angka 0. Ada juga model yang memiliki *density* yang tinggi yaitu model dengan fungsi aktivasi Sigmoid yang berarti model mengalami *vanishing gradient* di mana model stagnan karena tidak mengetahui arah untuk memperbaiki model.

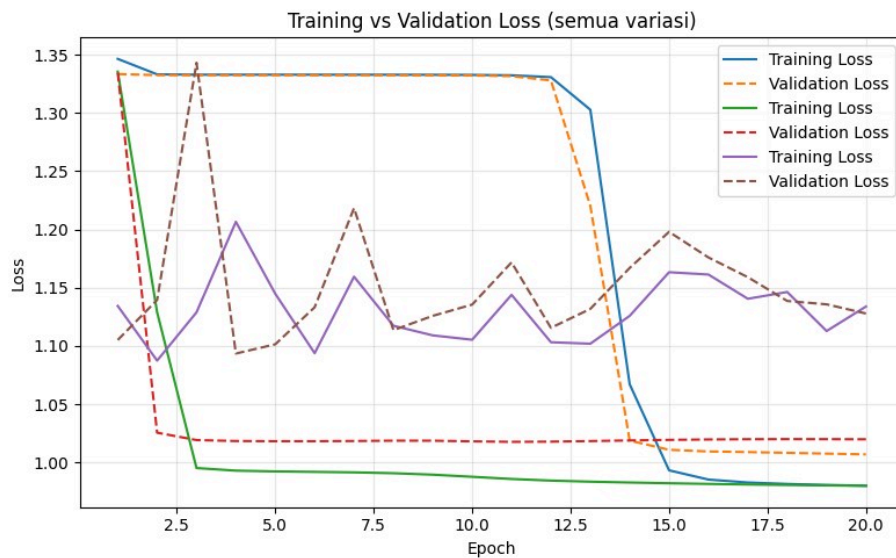
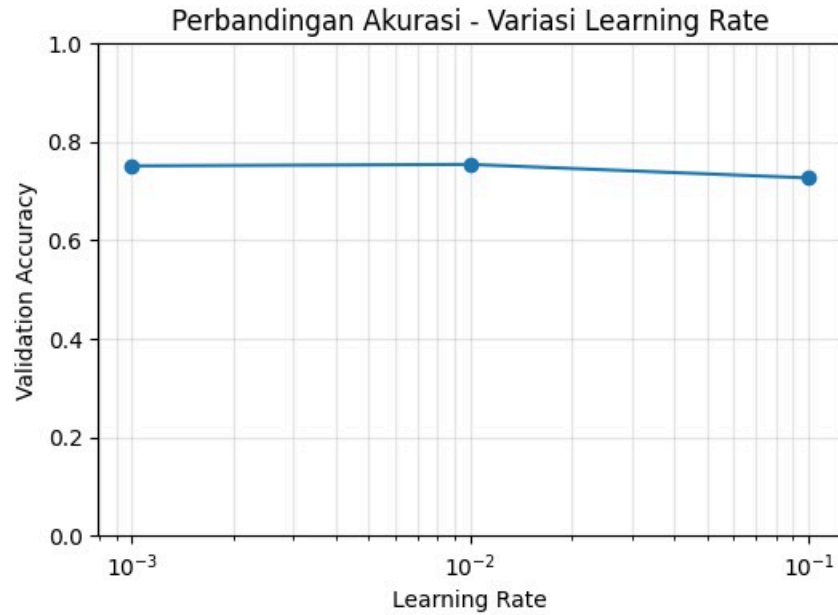
2.2.3. Pengaruh Learning Rate

Variasi Learning rate yang diujikan ialah 0.001, 0.01, dan 0.1. Didapatkan hasil akurasi dari variasi learning rate sebagai berikut:

Learning Rate	Akurasi
0.001	0.7510
0.01	0.7540
0.1	0.7265

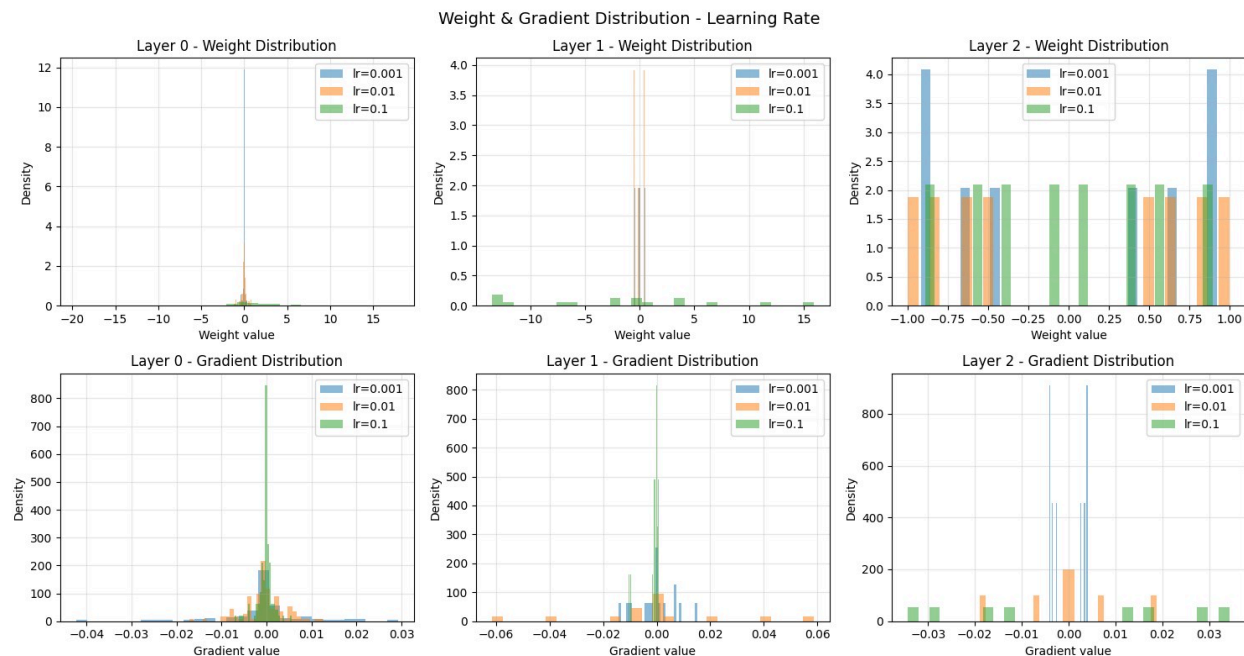
Terlihat bahwa learning rate 0.01 menghasilkan akurasi yang paling tinggi. Pada learning rate 0.001, angka learning rate yang terlalu kecil membuat model lebih lama untuk melakukan *training*. Pada learning rate 0.1, angka learning rate terlalu besar hingga muncul saat-saat dimana angka loss yang seharusnya mengecil malah membesar.

Didapatkan grafik perbandingan akurasi dan grafik training loss dan validation loss sebagai berikut:



Secara berurut legenda dari atas ke bawah: 0.001, 0.01, dan 0.1. Terlihat bahwa training loss pada model dengan learning rate 0.01, garis berwarna hijau, menunjukkan hasil terbaik karena mampu konvergen dengan cepat dan stabil di epoch awal-awal. Garis berwarna biru yang menandakan model dengan learning rate 0.001 sempat stagnan di awal-awal sebelum turun di kisaran epoch 13. Garis berwarna ungu, model dengan learning rate 0.1, mengalami fluktuasi sepanjang epoch yang menandakan model *overshoot* titik optimal sehingga nilai loss fluktuasi. Akibatnya, bobot model terus berubah dan tidak bisa menetap.

Selain itu, didapatkan grafik distribusi bobot dan gradien bobot dari beberapa/semua layer pada model sebagai berikut:



Terlihat bahwa *weight distribution* di layer 2 lebih merata dan menyebar dibandingkan layer 0 dan 1 untuk learning rate 0.1 dan 0.01. Untuk learning rate 0.001, distribusi *weight* mendekati 1 dan -1. *Weight distribution* yang merata, seperti pada learning rate 0.1 dan 0.01, menandakan diversitas fitur dan setiap neuron tidak atau sangat sedikit yang redundan. Learning rate 0.01 lebih tersebar dan lebih diversitas dibandingkan learning rate 0.1. *Weight Distribution* yang tidak merata, pada learning rate 0.001, menandakan neuron banyak yang redundan karena *weight distribution* berdiam di dekat angka 1 dan -1.

Gradient distribution cukup merata di layer 2 untuk learning rate 0.1 dan 0.01 dengan learning rate 0.1 lebih merata dan menyebar. Learning rate 0.001 berdiam di kisaran angka 0. *Gradient distribution* yang merata, pada learning rate 0.1 dan 0.01, menandakan model aktif melakukan update *weight* dengan skala yang bervariasi. Terlihat bahwa learning rate 0.1 lebih tersebar dibandingkan 0.01 yang berarti model dengan learning rate 0.1 lebih aktif dibandingkan 0.01. Model dengan learning rate 0.001 memiliki distribusi gradien yang terpusat di angka 0. Hal tersebut menandakan *Vanishing gradient* di mana model stagnan karena tidak mengetahui arah untuk memperbaiki model.

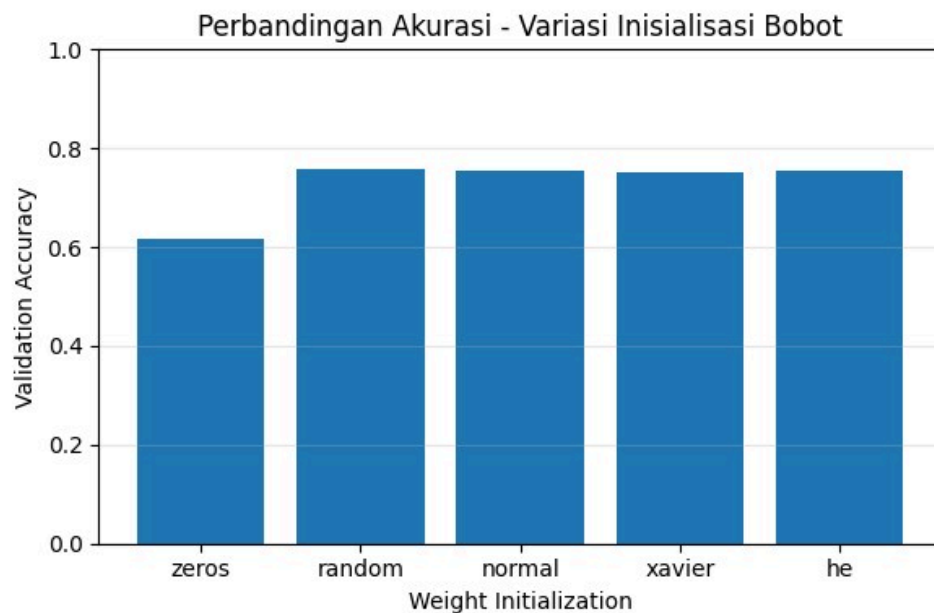
2.2.4. Pengaruh Inisialisasi Bobot

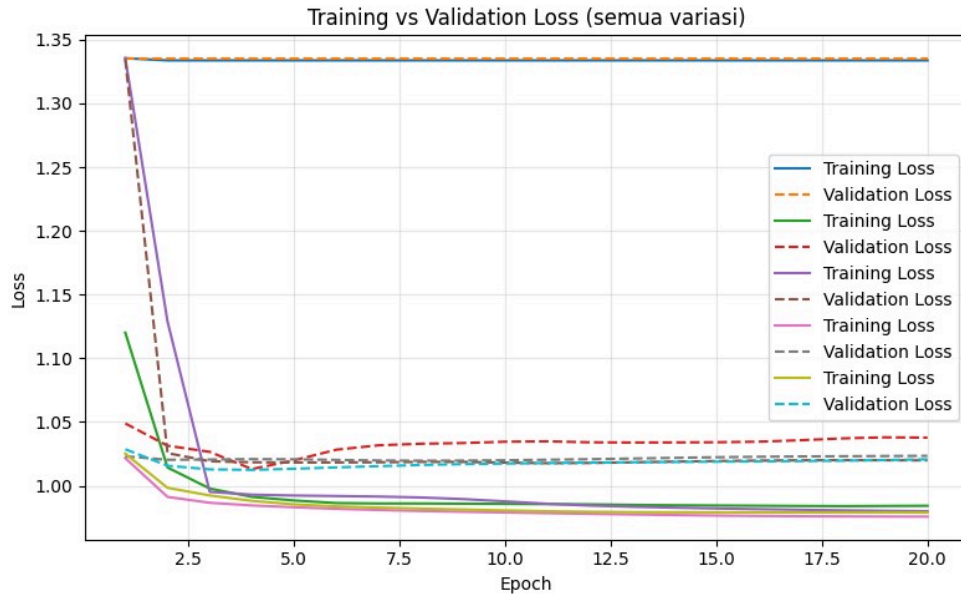
Variasi Initial weight yang diujikan adalah Zeros/Kosong, Random, Xavier, He. Didapatkan hasil akurasi dari variasi inisialisasi bobot sebagai berikut

Initial Weight	Akurasi
Zeros	0.6155
Random	0.7570
Normal	0.7540
Xavier	0.7510
He	0.7555

Terlihat bahwa variasi initial *weight* Random memberikan nilai akurasi yang paling tinggi. Perbedaan yang jauh terlihat pada variasi initial *weight* kosong. Hal tersebut cukup wajar dikarenakan tanpa *weights*, model tidak mampu mempelajari fitur yang berbeda-beda sehingga tidak memberikan akurasi yang tepat. Terlihat juga bahwa model memprediksi semua dataset sebagai 'Placed' yang menandakan model gagal di-*train* sepenuhnya.

Didapatkan grafik perbandingan akurasi dan grafik training loss dan validation loss sebagai berikut:





Secara berurut legenda dari atas ke bawah: Zeros, Random, Normal, Xavier, He. Hampir semua proses training semua model berjalan dengan lancar karena mampu konvergen di epoch awal. Satu model yang gagal konvergen adalah model dengan inisialisasi bobot Zero. Artinya, model tersebut gagal total sehingga angka loss tetap sangat besar hingga akhir epoch 20. Hal tersebut bisa disebabkan oleh inisialisasi weight kosong dan model gagal *training*.

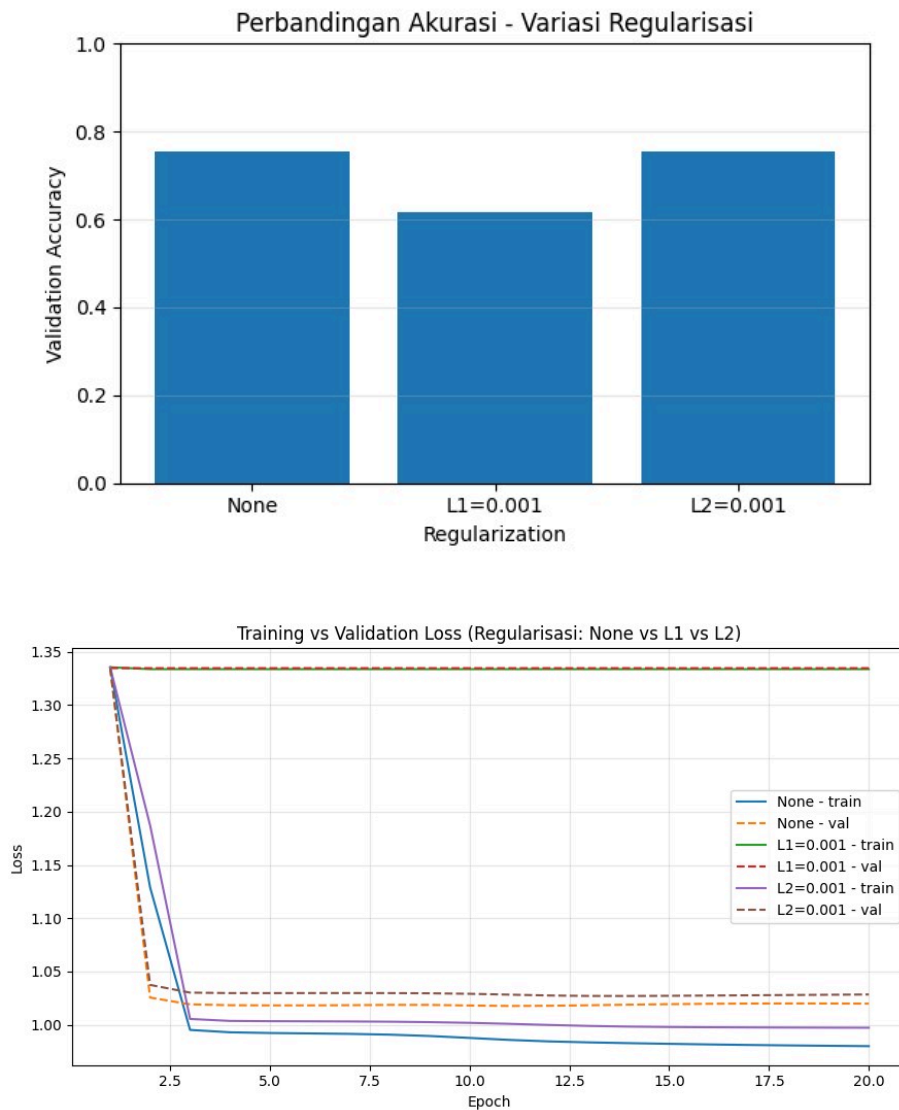
2.2.5. Pengaruh Regularisasi

Regularisasi adalah teknik yang digunakan untuk mencegah overfitting dengan menambahkan penalti pada fungsi loss berdasarkan besarnya nilai bobot model. Teknik ini bertujuan untuk membatasi kompleksitas model agar memiliki kemampuan generalisasi yang lebih baik pada data baru. Dalam laporan ini, diujikan dua jenis regularisasi: L1 (Lasso) yang menambahkan nilai absolut bobot dan L2 (Ridge) yang menambahkan kuadrat dari bobot ke dalam perhitungan loss. Didapatkan hasil akurasi dari variasi regularisasi sebagai berikut:

Regularization	Akurasi
None	0.7540
L1	0.6155
L2	0.7560

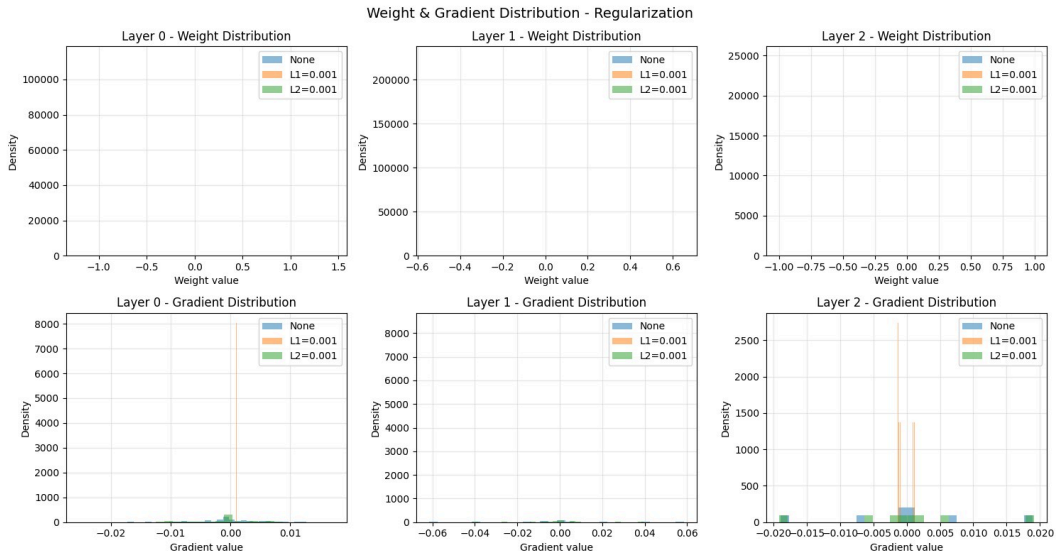
Terlihat bahwa Regularization None dan L2 berdiam di angka yang dekat tetapi Regularization L1 berdiam di angka 0.6155 yang menandakan bahwa L1 terjadi *underfitting* dimana model tidak mampu merepresentasikan data dengan benar.

Didapatkan grafik perbandingan akurasi dan grafik training loss dan validation loss sebagai berikut:



Terlihat bahwa Regularization L1 stagnan sedangkan Regularization None dan L2 tidak stagnan dan serupa. L1 yang stagnan menandakan model gagal *training*.

Selain itu, didapatkan grafik distribusi bobot dan gradien bobot dari beberapa/semua layer pada model sebagai berikut:



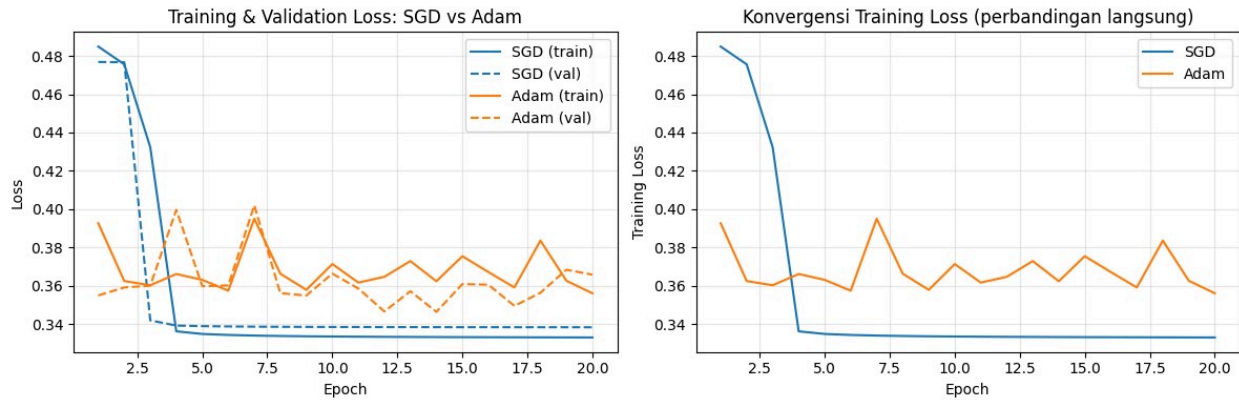
Terlihat gradient distribution untuk regularisasi None dan L2 cukup merata tetapi regularisasi L1 berdiam di tengah dengan angka yang sangat besar. Pada L1 terjadi *Vanishing gradient* di mana model stagnan karena tidak mengetahui arah untuk memperbaiki model.

2.2.6. Perbandingan Dengan Library sklearn

Model FFNN from *scratch* menggunakan konfigurasi layer $[X_train.shape[1], 4, 4, 2]$ dan fungsi aktivasi tanh, tanh, softmax. Model FFNN dari library sklearn menggunakan konfigurasi yang menyerupai. Hasilnya didapatkan akurasi FFNN from *scratch* 0.7570 dan akurasi sklearn 0.7500. Walaupun perbedaan angka akurasi tidak jauh, implementasi FFNN from *scratch* menghasilkan angka akurasi yang lebih bagus dibandingkan FFNN dari library sklearn.

2.2.7. Perbandingan Gradient Descent Standar Dengan Adam

Perbandingan Gradient Descent Standar dan Adam dilihat dari kecepatan konvergensi. Didapatkan perbedaan grafik sebagai berikut:



Adam optimizer memulai proses *training* dengan angka loss yang lebih kecil dibandingkan Gradient Descent. Hal tersebut terjadi karena Adam optimizer menggunakan *Adaptive Moment Estimation* yang menghitung *learning rate* untuk setiap parameter sehingga lebih efisien, berbeda dengan gradient descent yang membutuhkan beberapa iterasi sebelum menemukan descent. Walaupun begitu, Gradient Descent lebih stabil sepanjang epoch dibandingkan adam optimizer yang naik turun membentuk *zigzag*. Adam optimizer membuat pola *zigzag* karena adam sering *overshoot* dari *global minimum* yang membuatnya naik turun. Waktu total yang dibutuhkan Gradient Descent sebanyak 53.25 detik sedangkan waktu total yang dibutuhkan Adam Optimizer sebanyak 53.52 detik. Hasil akhir Gradient Descent menghasilkan akurasi 0.49 sedangkan adam optimizer menghasilkan nilai akurasi 0.2810. Hasil akurasi kedua model sangat jauh walaupun teori mengatakan bahwa adam optimizer seharusnya menghasilkan nilai akurasi yang serupa. Hal tersebut bisa terjadi akibat beberapa hal seperti *hyperparameter* default adam yang tidak cocok dengan dataset, *overshooting* yang terlalu ekstrem, dan lain-lain.

BAB III

KESIMPULAN DAN SARAN

3.1. Kesimpulan

Berdasarkan pengujian yang telah dilakukan, didapatkan model-model yang memberikan nilai akurasi terbaik ialah model dengan 1 layer, model dengan 8 neuron, model dengan fungsi aktivasi linear, model dengan learning rate 0.01, model dengan initial weight random, model dengan regularisasi L2, dan model dengan Gradient Descent SGD. Walaupun masing-masing model memberikan nilai akurasi terbaik, pembuatan model dengan *a little bit from everything* tidak menjamin model yang sangat kuat. Tuning parameter-parameter tertentu mampu mengubah nilai akurasi akhir. Memaksimalkan semua angka dalam parameter bukan strategi tuning yang *viable* dikarenakan model bisa saja malah *overshot* dari prediksi yang sebenarnya.

Untuk model FeedForward Neural Network sendiri, model berjalan sangat lambat apalagi jika menggunakan hyperparameter ekstrim seperti epoch yang banyak, fungsi aktivasi yang berat, ukuran layer yang sangat besar, dan lain-lain. Bahkan pada pengujian, yang tidak menggunakan tuning ekstrim, memakan waktu hingga setengah jam untuk menyelesaikan eksperimen. Tidak hanya itu, tuning-tuning tertentu bisa menyebabkan model untuk *Collapse* sehingga meng-*set* semua data menjadi positif atau negatif sesuai dengan rata-rata mayoritas.

Walaupun begitu, model FFNN merupakan model yang cukup bagus untuk digunakan dalam pembelajaran mesin. Hanya saja dibutuhkan waktu, kesabaran, dan kepintaran dalam *men-tune* serta mengimplementasikan model FFNN itu sendiri.

3.2. Saran

Implementasi model FFNN yang lebih stabil dan lebih cepat sangat dianjurkan karena waktu yang dibutuhkan untuk melakukan training amat sangat lama. Alternatif bahasa pemrograman juga bisa menjadi pilihan seperti integrasi dengan C++ yang secara performa lebih cepat daripada python.

REFERENSI

- Kingma, D. P., & Ba, J. L. (2014, December 22). Adam: A method for stochastic optimization. *UvA-DARE (University of Amsterdam)*. <https://doi.org/10.48550/arXiv.1412.6980>
- Lu, L., Su, Y., & Karniadakis, G. E. (2018). Collapse of deep and narrow neural nets. [doi:10.48550/ARXIV.1808.04947](https://doi.org/10.48550/ARXIV.1808.04947)
- Osajima, J. (n.d.). The math behind Neural Networks - forward propagation. The Math behind Neural Networks - Forward Propagation | Jason {osa-jima}. <https://www.jasonosajima.com/backwardprop>
- Osajima, J. (n.d.). The math behind Neural Networks - forward propagation. The Math behind Neural Networks - Forward Propagation | Jason {osa-jima}. <https://www.jasonosajima.com/forwardprop>
- Sazlı, M. H. (2006). A brief review of feed-forward neural networks. Communications Faculty of Sciences University of Ankara Series A2-A3 Physical Sciences and Engineering, 50(01). https://doi.org/10.1501/commua1-2_0000000026

LAMPIRAN

1. Link repo: <https://github.com/Aramazaya/Aram-Intelligence-3.0>
2. Pembagian tugas tiap anggota kelompok

Nama	NIM	Tugas
Sebastian Hung Yansen	13523070	Laporan, Pengujian, Implementasi Adam Optimizer, Notebook
Aramazaya	13523082	FFNN Model, Autograd, Xavier&He Weight, Leaky ReLU & eLU

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Sebastian Hung Yansen/13523070, Aramazaya/13523082

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas : Feedforward Neural Network

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

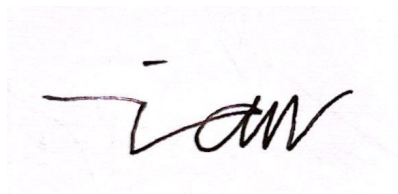
Alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	No AI.
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	AI-assisted workflow.
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	AI-assisted idea generation and structuring.

Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	No AI.
Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Maret 2026



Sebastian Hung Yansen

13523070



Aramazaya

13523082